

John the Ripper 분석

송민호

유튜브: <https://youtu.be/q20tJEpB9PQ>

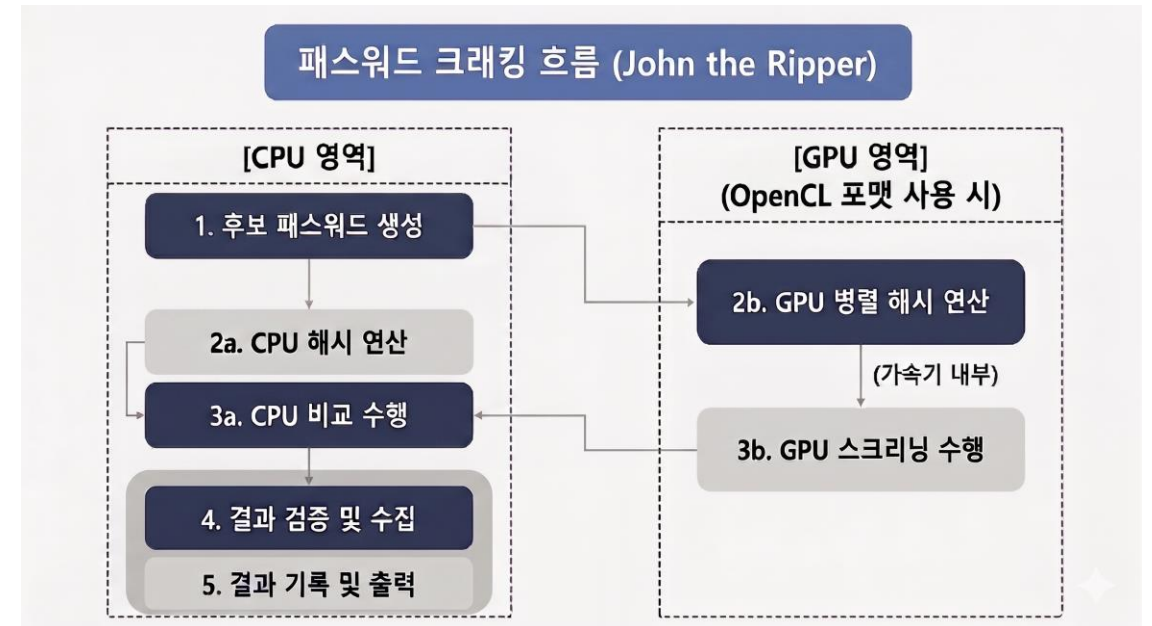
John the Ripper

- 패스워드 크래킹 도구는 공격 방식과 사용 목적에 따라 다양한 형태로 존재함
 - John the Ripper는 GPU 및 SIMD 기반 연산 가속기를 적극 활용하는 해시 크래킹 도구

도구	주요 목적	공격 대상	공격 방식	특징	가속기 지원
John the Ripper	password hash cracking	Unix password hash, 다양한 hash	Dictionary, incremental brute-force	오래된 전통적인 cracking tool, 확장성 좋음	SIMD / OpenMP 기반 CPU 최적화, 일부 GPU 지원

John the Ripper 동작 흐름

- CPU에서 후보 패스워드를 생성 후 선택한 포맷에 전달
 - 포맷: 해시 알고리즘별 플러그인. CPU 구현과 OpenCL 구현으로 나뉘는 경우가 많음
- 선택한 포맷에 따라 해시 연산 수행
 - CPU 포맷: CPU 기반 해시 연산 수행
 - OpenCL 포맷: GPU kernel 기반 해시 연산 수행
- 목표 해시 비교 수행
 - 해시 비교는 CPU에서 진행
 - GPU 스크리닝: 일부 포맷은 커널 안에서 비트맵, 부분 일치 등으로 후보를 줄인 뒤 CPU에 넘겨 비교 수행
- 일치한 결과 기록 및 출력 수행



John the Ripper는 포맷 기반 구조를 사용하여 CPU와 GPU 역할이 포맷에 따라 달라질 수 있음

CPU 역할	GPU 역할 (OpenCL 포맷 사용 시)
후보 패스워드 생성	병렬 해시 계산
포맷 제어 및 결과 검증	GPU 스크리닝 수행 가능
결과 기록 및 출력	대량 병렬 처리

John the Ripper의 연산 검증 구조

• 런타임 전

- Self-test(KAT) 수행

- 포맷 공통 Self-test 구조 사용
- 미리 정의된 입력, 해시값 비교를 통해 정상 동작 여부 확인

• 런타임 중

- 해시 계산 과정에 대한 무결성 검증 없음
- 포맷이 반환한 연산 결과로 정답 여부 확인

• 런타임 후

- 결과 재계산·교차 검증 미수행
- 일치한 결과를 potfile 및 출력으로 기록

단계	검증 여부	내용
런타임 전	o	Self-test(KAT)
런타임 중	x	추가 무결성 검증 없이 결과 사용
런타임 후	x	결과 재검증 없음

John the Ripper는 포맷별로 실행 전 정상 동작을 Self-test로 확인하지만 런타임 중·후의 연산 무결성 검증은 수행하지 않음

John the Ripper의 사전 연산 검증: Self-test

- Self-test는 선택된 포맷의 해시 연산의 정상 동작 여부를 확인하기 위한 KAT(Known Answer Test) 기반 검증 과정
 - John/src/formats.c에서 진행
- 실제 크래킹 작업 수행 전에 Self-test 진행
 - Tests[]: 포맷별로 정의된 기준 평문, 해시값
 - 미리 정의된 입력값, 해시값 비교를 통해 연산 정상 동작 여부 확인
- CPU 포맷과 OpenCL 포맷은 동일한 Self-test 프레임워크 사용
 - 검증 대상이 되는 해시 연산 구현은 포맷마다 상이함

```
#define FORMAT_TAG          "$dynamic_0$"
#define TAG_LENGTH          (sizeof(FORMAT_TAG) - 1)
#define FORMAT_TAG2         "{MD5}"
#define FORMAT_TAG2_LEN     (sizeof(FORMAT_TAG2) - 1)

static struct fmt_tests tests[] = {
    {"5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {FORMAT_TAG "5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {"098f6bcd4621d373cade4e832627b4f6", "test"},
    {"098F6BCD4621D373CADE4E832627B4F6", "test"},
    {FORMAT_TAG "378e2c4a07968da2eca692320136433d", "thatsworking"},
    {FORMAT_TAG "8ad8757baa8564dc136c1e07507f4a98", "test3"},
    {"d41d8cd98f00b204e9800998ecf8427e", ""},
    {"c4ca4238a0b923820dcc509a6f75849b", "1"},
    {"c20ad4d76fe97759aa27a0c99bfb6710", "12"},
    {"202cb962ac59075b964b07152d234b70", "123"},
    {"81dc9bdb52d04dc20036dbd8313ed055", "1234"},
    {"827ccb0eea8a706c4c34a16891f84e7b", "12345"},
}
```

CPU 기반 MD5

```
static unsigned int shift64_ht_sz, shift64_ot_sz;

static unsigned int key_idx = 0;
static struct fmt_main *self;

#define MIN_KEYS_PER_CRYPT 1
#define MAX_KEYS_PER_CRYPT 1

static struct fmt_tests tests[] = {
    {"5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {FORMAT_TAG "5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {"098f6bcd4621d373cade4e832627b4f6", "test"},
    {FORMAT_TAG "378e2c4a07968da2eca692320136433d", "thatsworking"},
    {FORMAT_TAG "8ad8757baa8564dc136c1e07507f4a98", "test3"},
    {"d41d8cd98f00b204e9800998ecf8427e", ""},
}
```

OpenCL 기반 MD5

John the Ripper Self-test 구조

- Self-test는 실제 크래킹과 동일한 포맷 함수 흐름으로 수행

- **fmt_self_test()**에서 Self-test 수행

- Self-test 진입 함수
- 테스트용 버퍼 및 상태 초기화
- fmt_self_test_body() 호출

- **fmt_self_test_body()**

- tests[] 기반 테스트 벡터 순회
- 실제 포맷 함수(crypt_all, cmp_*) 사용
- crypt_all()을 통해 테스트 벡터 기반 해시 연산 수행
- cmp_*(())를 통해 계산 결과 비교 수행

```
char *fmt_self_test(struct fmt_main *format, struct db_main *db)
{
    char *retval;
    void *binary_alloc, *salt_alloc;
    void *binary_copy, *salt_copy;

    binary_copy = alloc_binary(&binary_alloc,
        format->params.binary_size?format->params.binary_size:1, format->params.binary_align);
    salt_copy = alloc_binary(&salt_alloc,
        format->params.salt_size?format->params.salt_size:1, format->params.salt_align);

    self_test_running = 1;

    retval = fmt_self_test_body(format, binary_copy, salt_copy, db, benchmark_level);

    self_test_running = 0;

    MEM_FREE(salt_alloc);
    MEM_FREE(binary_alloc);

    return retval;
}
```

```
static char *fmt_self_test_body(struct fmt_main *format,
    void *binary_copy, void *salt_copy, struct db_main *db, int full_lvl)
{
    static char s_size[200];
```

OpenCL 포맷의 경우 crypt_all() 내부에서 실제 GPU backend 사용 가능

Self-test 실패 동작 분석

- Self-test용 기준 해시값을 비정상 값으로 수정할 경우
 - “FAILED (valid (before init))” 발생 후 John the Ripper 실행 중단
 - CPU 및 OpenCL 포맷에서 동일하게 Self-test 실패 확인

```
#define FORMAT_TAG        "$dynamic_0$"
#define TAG_LENGTH        (sizeof(FORMAT_TAG) - 1)
#define FORMAT_TAG2        "{MD5}"
#define FORMAT_TAG2_LEN    (sizeof(FORMAT_TAG2) - 1)

static struct fmt_tests tests[] = {
    {"5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {FORMAT_TAG "5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {"098f6bcd4621d373cade4e832627b4f6", "test"},
    {"098F6BCD4621D373CADE4E832627B4F6", "test"},
    {FORMAT_TAG "378e2c4a07968da2eca692320136433d", "thatsworking"},
    {FORMAT_TAG "8ad8757baa8564dc136c1e07507f4a98", "test3"},
    {"d41d8cd98f00b204e9800998ecf8427e", ""},
    {"c4ca4238a0b923820dcc509a6f75849b", "1"},
    {"c20ad4d7fe97759aa27a0c99bfff6710", "12"},
    {"202cb962ac59075b964b07152d234b70", "123"},
    {"81dc9bdb52d04dc20036dbd8313ed055", "1234"},
    {"827ccb0eea8a706c4c34a16891f84e7b", "12345"},
}
```

```
static struct fmt_tests tests[] = {
    {"5a105e8b9d40e1329780d62ea2265d8a.", "test1"},
    {FORMAT_TAG "5a105e8b9d40e1329780d62ea2265d8a", "test1"},
}
```

해시 문자열 뒤에 "." 추가
→ 비정상 형식으로 변경

```
make[1]: `../run/unique' is up to date.
make[1]: `../run/undrop' is up to date.
make[1]: `../run/rar2john' is up to date.
make[1]: `../run/zip2john' is up to date.
make[1]: `../run/gpg2john' is up to date.
make[1]: `../run/base64conv' is up to date.
```

Make process completed.

```
Testing: Raw-MD5 [MD5 128/128 ASIMD 4x2]... FAILED (valid (before init))
```

John the Ripper 실행 중단 확인

Self-test 실패 동작 분석

- Self-test용 기준 해시값을 실제 계산 결과와 일치하지 않는 값으로 수정할 경우
 - “FAILED (cmp_exact())” 발생 후 John the Ripper 실행 중단
 - CPU 및 OpenCL 포맷에서 동일하게 Self-test 실패 확인

```
#define FORMAT_TAG        "$dynamic_0$"
#define TAG_LENGTH        (sizeof(FORMAT_TAG) - 1)
#define FORMAT_TAG2        "{MD5}"
#define FORMAT_TAG2_LEN    (sizeof(FORMAT_TAG2) - 1)

static struct fmt_tests tests[] = {
    {"5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {FORMAT_TAG "5a105e8b9d40e1329780d62ea2265d8a", "test1"},
    {"098f6bcd4621d373cade4e832627b4f6", "test"},
    {"098F6BCD4621D373CADE4E832627B4F6", "test"},
    {FORMAT_TAG "378e2c4a07968da2eca692320136433d", "thatsworking"},
    {FORMAT_TAG "8ad8757baa8564dc136c1e07507f4a98", "test3"},
    {"d41d8cd98f00b204e9800998ecf8427e", ""},
    {"c4ca4238a0b923820dcc509a6f75849b", "1"},
    {"c20ad4d7fe97759aa27a0c99bfff6710", "12"},
    {"202cb962ac59075b964b07152d234b70", "123"},
    {"81dc9bdb52d04dc20036dbd8313ed055", "1234"},
    {"827ccb0eea8a706c4c34a16891f84e7b", "12345"},

```

```
static struct fmt_tests tests[] = {
    {"5a105e8b9d40e1329780d62ea2265d8b", "test1"},
    {FORMAT_TAG "5a105e8b9d40e1329780d62ea2265d8a", "test1"},

```

해시 문자열 마지막을 “a”에서 “b”로 수정
→ 일치하지 않는 값으로 변경

```
rm -f ../run/zip2john
rm -f ../run/gpg2john
ln -s john ../run/rar2john
ln -s john ../run/zip2john
ln -s john ../run/gpg2john
rm -f ../run/base64conv
ln -s john ../run/base64conv
```

```
Make process completed.
Testing: Raw-MD5 [MD5 128/128 ASIMD 4x2]... FAILED (cmp_exact(0))
```

John the Ripper 실행 중단 확인

John the Ripper 결과 처리 구조

- John the Ripper는 단계별 비교 함수를 통해 후보를 점진적으로 필터링하며 최종 일치 여부를 판단함
- **cmp_all()**
 - 여러 후보 결과를 빠르게 확인하는 1차 스크리닝
 - 목표 해시와 일치 가능성이 있는 후보 존재 여부 확인
 - 포맷 별로 목표 전체 또는 일부 워드를 비교하여 다음 단계로 넘길지 판단함
- **cmp_one()**
 - cmp_all() 통과 후 후보 값들의 인덱스 단위로 비교 수행
 - 특정 후보 인덱스가 목표 해시와 일치하는지 확인
- **cmp_exact()**
 - 최종 완전 일치 여부 검증
 - 이전 단계에서 일부만 비교한 포맷은 여기서 추가 확인 수행 가능

```
if ((match && !format->methods.cmp_all(binary, match)) ||  
    (!match && format->methods.cmp_all(binary, match))) {  
    if (options.verbosity > VERB_LEGACY)  
        snprintf(err_buf, sizeof(err_buf), "cmp_all(%d) %s", match, ciphertext);  
    else  
        sprintf(err_buf, "cmp_all(%d)", match);  
    return err_buf;  
}
```

```
/* Presumably no index mapping occurred, so require match at input index */  
i = -1;  
if (format->methods.cmp_one(binary, index))  
    i = index;  
} else {  
    /* Index mapping likely occurred, so find where the match is */  
    for (i = match - 1; i >= 0; i--) {  
        if (format->methods.cmp_one(binary, i))  
            break;  
    }  
}
```

```
if (!format->methods.cmp_exact(ciphertext, i)) {  
    if (options.verbosity > VERB_LEGACY)  
        snprintf(err_buf, sizeof(err_buf), "cmp_exact(%d) %s", match, ciphertext);  
    else  
        sprintf(err_buf, "cmp_exact(%d)", i);  
    return err_buf;  
}
```

John the Ripper는 런타임 후 무결성 검증은 수행하지 않음

크래킹 툴 연산 검증 비교

- **Self-test 기반 검증**

- Hashcat과 John the Ripper 모두 실행 전 **Self-test(KAT)**를 통해 **정상 동작 여부 확인**
- 실제 크랙과 동일한 연산 경로를 사용하여 기준 입력값과 해시값 비교 수행

- **한계점**

- Self-test는 **실행 전 정상 동작 여부만 확인**
- 런타임 중 발생 가능한 Bit-flip, fault injection, 메모리 오류 등에 대한 **지속적 무결성 검증 구조는 제한적**
- Hashcat과 John the Ripper 모두 **해시 비교를 수행**하지만 연산 과정 자체의 **무결성을 재검증하는 구조는 아님**
- 런타임 중 연산 오류 발생 시 정상 결과가 실패로 판단되거나(**False Negative**) 비정상 결과가 일치로 판단될(**False Positive**) 가능성 존재

Q & A